# KEY — Biweekly Assessment BW03 — Weeks 5–6 (Trainer only)

**Do not share with students.** Excluded from student pack (`*_KEY.md`).

## A. MCQ answers

1. **a** — A named block you can call
2. **a** — Same name, different parameter lists
3. **b** — The JVM entry point
4. **b** — Creates an object and runs the matching constructor
5. **a** — Must match the class name
6. **c** — The current object
7. **b** — Is no longer provided
8. **b** — Unique per object
9. **c** — One shared copy for the class
10. **b** — Cannot compile `acct.balance` — must use methods
11. **b** — It lets callers skip `deposit` / `withdraw` and smash the rule
12. **b** — Two names point at the same object

## B. Coding — model points

1. `LineItem` with constructor + instance `lineTotal` + static `withGst`. Tea ₹15 × 2 = `30.0`; GST → `31.5`. Award marks for **instance vs static** used correctly. No `extends`.

```
public class LineItem {
    String name;
    double price;

    LineItem(String name, double price) {
        this.name = name;
        this.price = price;
    }

    double lineTotal(int qty) {
        return price * qty;
    }

    static double withGst(double amount) {
        return amount * 1.05;
    }

    public static void main(String[] args) {
        LineItem tea = new LineItem("Tea", 15);
        double line = tea.lineTotal(2);
        System.out.println(line);
        System.out.println(withGst(line));
    }
}
```

2. Encapsulated **SecureBank** `BankAccount`. After +200 −100, balance `1100`; overdraft `5000` prints a message and leaves `1100`. `private` fields. No `setBalance`. `print` + `return` only.

```
public class BankAccount {
    private String holder;
    private double balance;

    BankAccount(String holder, double opening) {
        this.holder = holder;
        this.balance = opening;
    }

    void deposit(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount");
            return;
        }
        balance += amount;
    }

    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount");
            return;
        }
        if (amount > balance) {
            System.out.println("Insufficient funds");
            return;
        }
        balance -= amount;
    }

    double getBalance() {
        return balance;
    }

    public static void main(String[] args) {
        BankAccount a = new BankAccount("Anita", 1000);
        a.deposit(200);
        a.withdraw(100);
        a.withdraw(5000);
        System.out.println(a.getBalance());
    }
}
```

## C. Short answer — model

Encapsulation bundles data with the methods that use it and hides fields (`private`) so callers cannot smash rules. A public `balance` lets anyone write `acct.balance = -50000` and skip `withdraw`. `private` + `deposit` / `withdraw` keeps “money only changes through the door we validate.” Safer reports, one place to fix rules.

## Suggested scoring

| Section | Marks |
|---|---|
| A MCQ (1 each) | 12 |
| B Coding (6 each) | 12 |

| Section | Marks |
|---|---|
| C Short | 6 |
| **Total** | **30** |